Download the NDepend Trial and gain valuable insights into your .NET code within a few minutes

NDepend Blog

Improve your .NET code quality with NDepend

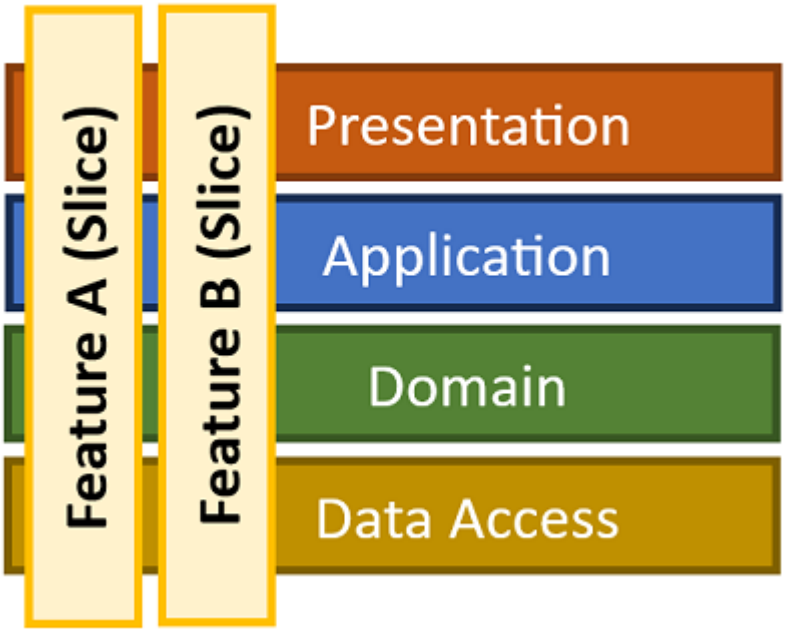
Vertical Slice Architecture in ASP.NET Core

June 10, 2025 6 minutes read

The organization of code in a solution is a subject of frequent debates. Currently, two prominent approaches have garnered attention: **Clean Architecture** versus **Vertical Slice**.

> **Clean Architecture** organizes code by **technical concerns** (e.g., UI, business logic, data access).
>
> **Vertical Slice Architecture** structures code by **feature or use case**, encapsulating all necessary layers per slice. It aims at minimizing coupling between unrelated features and strengthen cohesion within each feature.

This distinction is visually represented in the diagram below: layers are depicted as horizontal boxes, whereas features are represented as **vertical** boxes, hence the term *"Vertical Slice"*.



In this article, we will explore **Vertical Slice Architecture in ASP.NET Core** and discuss the benefits it offers for making our code more maintainable

Download the NDepend Trial and gain valuable insights into your .NET code within a few minutes

The Problems with Technical Layers

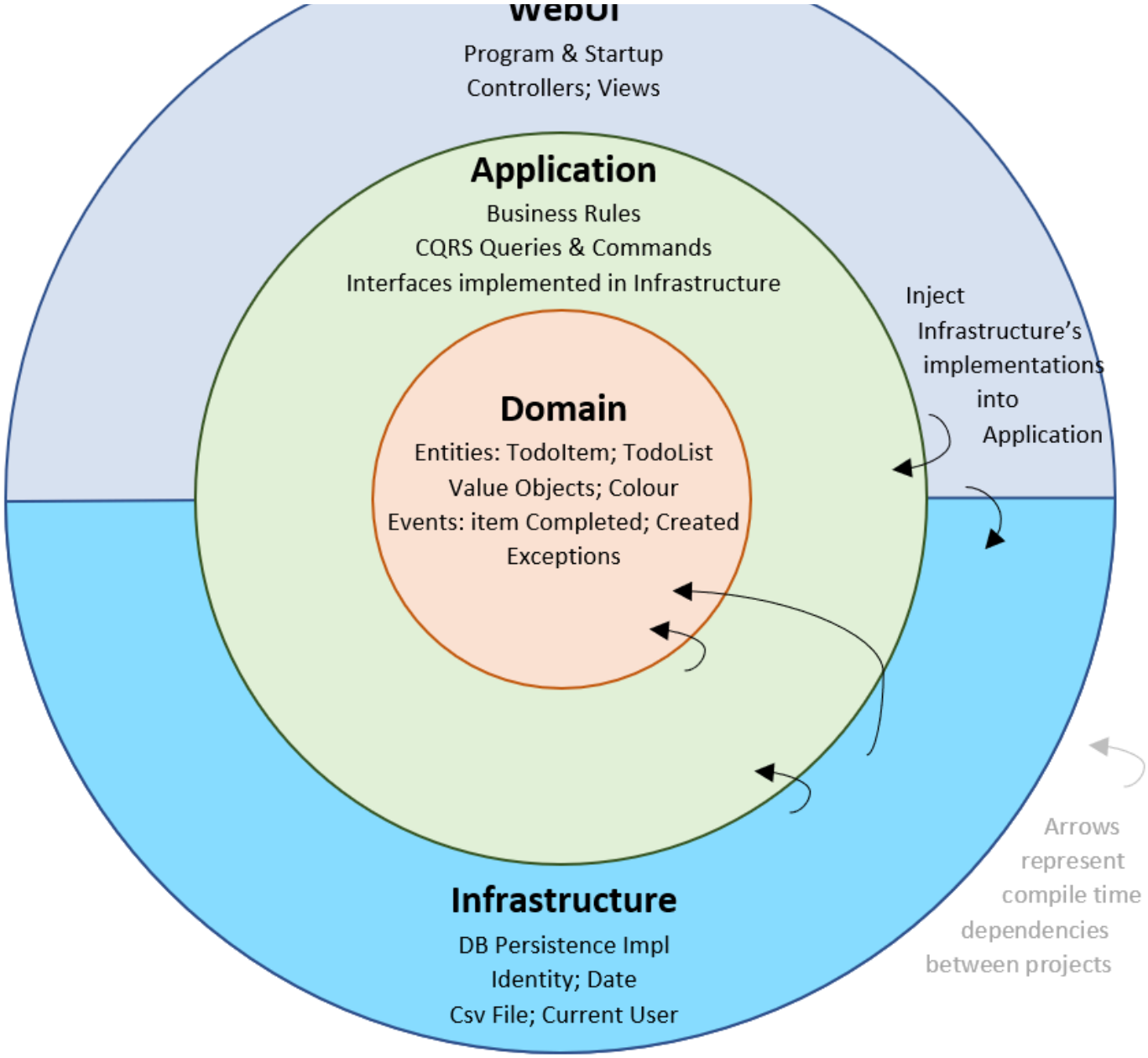
Historically, the layered approach preceded the slice approach. The slice approach represents an effort to address the two primary challenges that developers consistently encounter when working with layers:

Layer Coupling : The main constraint in a layered architecture arises from the tight coupling between its layers. Each layer must have an in-depth knowledge of the layers it depends on. This leads to a **rigid** architecture that becomes increasingly challenging to maintain over time.

Feature Fragmentation: Each unit of business functionality is distributed across multiple layers. This results in the inconvenience of navigating through multiple layers when introducing new features or improving existing ones.

It's worth highlighting that the original Clean Architecture approach, initially introduced by Robert C. Martin in 2012, is an attempt to address the first issue (Layers Coupling) through the Dependency Inversion Principle (DIP). We explain it in this post Clean Architecture in ASP.NET Core based on the Jason Taylor's Clean Architecture .NET solution template available here on github. This onion diagram provides a succinct summary of how layers should be coupled according to Uncle Bob (M.Martin) :

Download the NDepend Trial and gain valuable insights into your .NET code within a few minutes



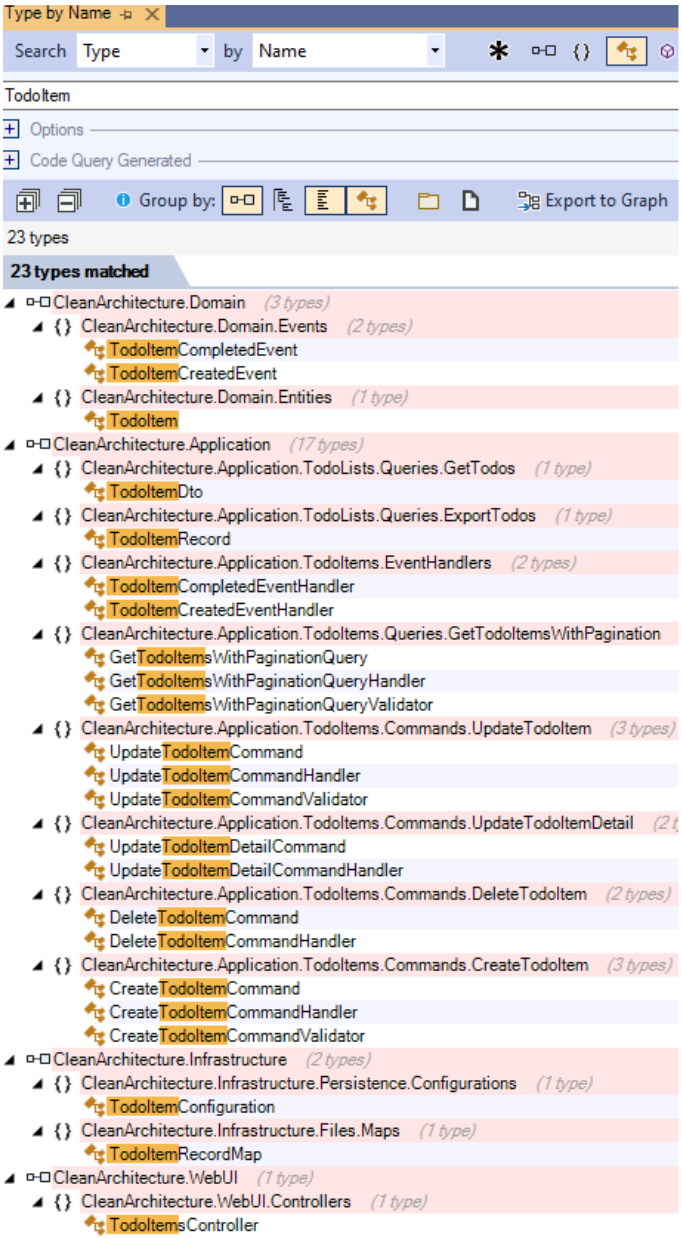
The Application layer is able to use all sorts of infrastructure (DB, Date, file, user profile...) thanks to abstractions whose implementations are injected from a higher layer, here from the Web UI layer. Doing so favors flexibility and reduces the overall rigidity of the solution. Coupling inversion is the gem of Clean Architecture. Yet the Feature Fragmentation problem still remains and is addressed by Vertical Slice.

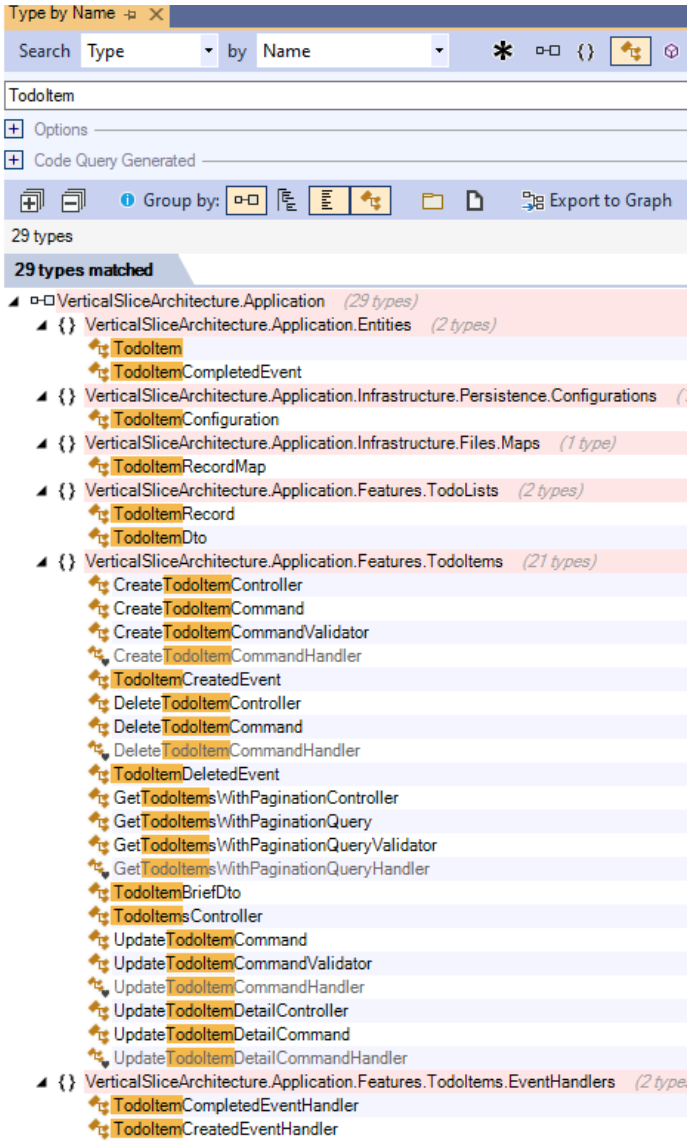
Benefits of Vertical Slice

Vertical Slice Architecture addresses Feature Fragmentation through **Code Locality**. It emphasizes the grouping of code associated with similar functionality, ideally within a single class, file, or package. This approach resonates with human preference for **neatly packaging-related elements in one cohesive unit**.

Nadir Badnjevic undertook the task of refactoring Jason Taylor's Clean Architecture .NET solution using Vertical Slices. Nadir's solution is available on github here. The domain is left untouched. For the purposes of clarity, the domain is minimal and only contains mostly one value object `Colour` and two entities: `TodoItem` and `TodoList`. If I search for implementations related to `TodoItem` in both solutions here is what I get:

Download the NDepend Trial and gain valuable insights into your .NET code within a few minutes





If we work on the `TodoItem` feature:

> With the Clean Architecture approach, we will touch potentially 13 namespaces spawn in 4 projects.
>
> With the Vertical Slice approach, we will touch potentially 6 namespaces all contained in a single project. Actually, we will mostly work within the feature namespace `Application.Features.TodoItems` that contains Queries, Commands, and Controllers related to this feature. This is the beauty of code locality!

Vertical Slice Architecture centers around the idea that artifacts that change together should be grouped together.

Impact of the Vertical Slice Approach

One key aspect of Vertical Slices is to **maintain slices independent from one another as much as possible**. Here is a comment on Jimmy Bogard's 2018 post on Vertical Slice Architecture that is worth reading:

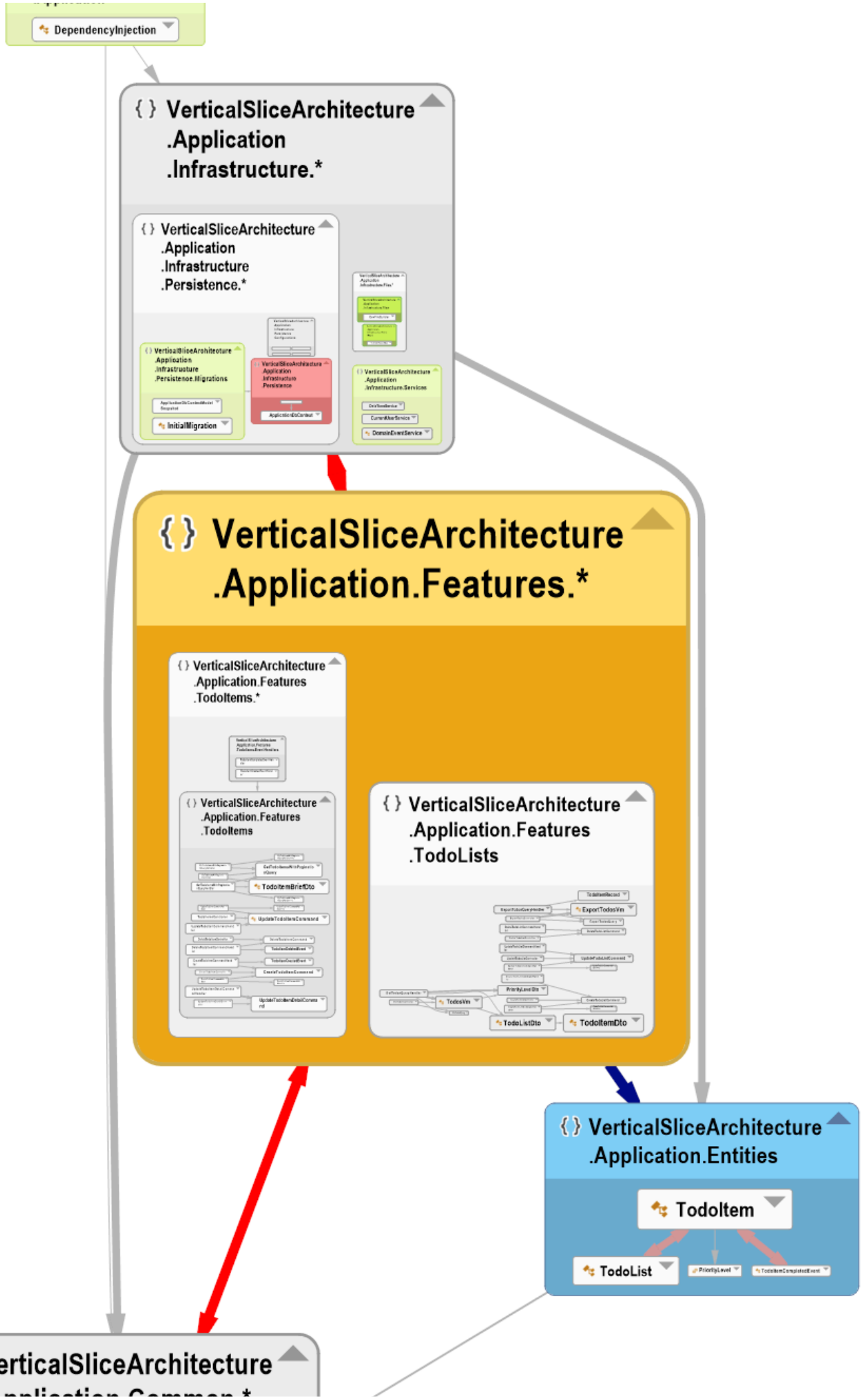
Download the NDepend Trial and gain valuable insights into your .NET code within a few minutes

over dogmatism, it still leaves me with a bad taste in my mouth. How do you handle this situations?"

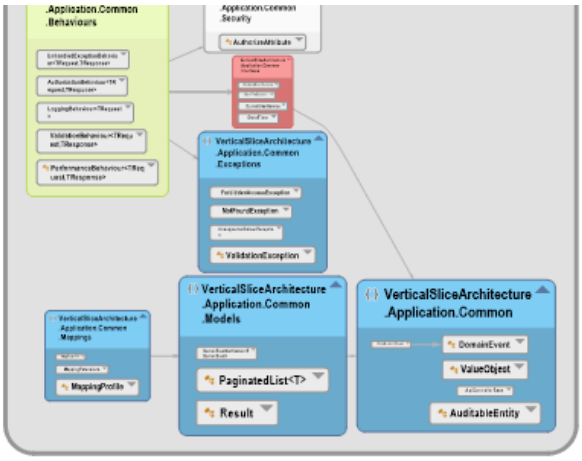
Jimmy Bogard "Refactor the common code to some shared service/location. It's really not different in any other architecture. ***Just don't call one feature from another****. Refactor!"*

See below a dependency graph (obtained with NDepend) of Nadir's solution. We can see that in the central `Features.*` namespace, `TodoItem` and `TodoList` implementations are independent of each other. This graph also shows that classes within each feature namespace are cohesive enough. This solution applies the principle of **high-cohesion low-coupling** at the feature level.

Download the NDepend Trial and gain valuable insights into your .NET code within a few minutes



Download the NDepend Trial and gain valuable insights into your .NET code within a few minutes



The features share the same domain model, else you might question if they should belong to the same solution. Nevertheless, the features do not inherently align with uniform infrastructure; each feature operates as an autonomous entity and possesses the freedom to leverage the most appropriate technologies and best practices. This **extra flexibility** is an important benefit of applying Vertical Slice. However, it leads to some concerning consequences:

> Applying Vertical Slice may lead to **code duplication** *to some extent*. This preference for duplication arises from a desire to avoid **awkward abstractions** that attempt to accommodate all scenarios without excelling in any specific one (we all know this phenomenon, aren't we?).
>
> In Clean Architecture, the abstractions of the infrastructure are inherently designed, though they might be implemented less straightforwardly. Such abstractions necessarily increase **testability**. When abiding by Vertical Slices these abstractions may be absent.
>
> Vertical Slice may **degrade code consistency**. Vertical Slice is sometimes compared to the **microservices** architecture, except that all services live in a shared solution. While the lack of code consistency between independent services is fine, it is annoying within a single solution because this **hinders the principle of least surprise**. Here is another interesting comment from the Jimmy Bogard post that summarizes this point:

"This is going to lead to more code bloat, more code duplication, more confusion among developers. Especially junior developers who are already easy enough to confuse when you don't have all these swirling options floating around. Why are we solving the same kind of problem, in the same application in multiple different ways? Why are we spending so much time on duplication of similar functionality? Monolithic applications have a lot of problems, but "this code is just too damned consistent" isn't one of them."

Conclusion

In this article, we have explored the concept of vertical slice architecture, which involves dividing the code into distinct *slices*, each representing a complete feature.

We've also discerned the contrast between vertical slice architecture and traditional layered architectural approaches that focuses on separating business rules from infrastructural concerns.

We've examined the advantages and disadvantages of both architectural styles. We can now conclude that **there is no one-size-fits-all approach**.

> Layers lead to more coupling and rigidity which can be really problematic in the long term.
>
> Slices lead to more duplication, reduced consistency, and potential testability glitches.

Download the NDepend Trial and gain valuable insights into your .NET code within a few minutes

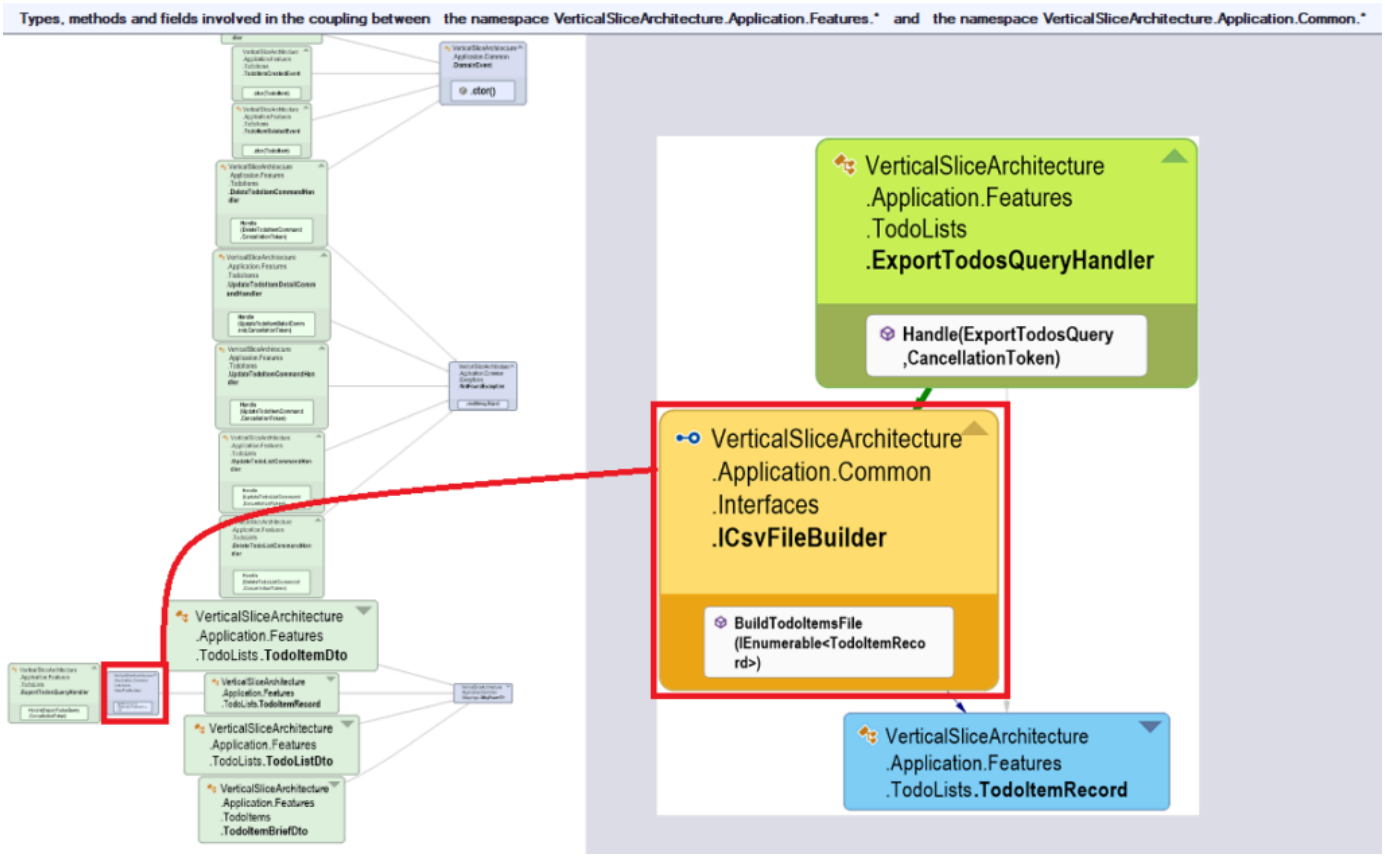
dependencies that won't limit your future options.

To learn more about the Vertical Slice approach I recommend the work of my friend Derek Comartin (CodeOpinion) who advocates for it through comprehensive videos and articles.

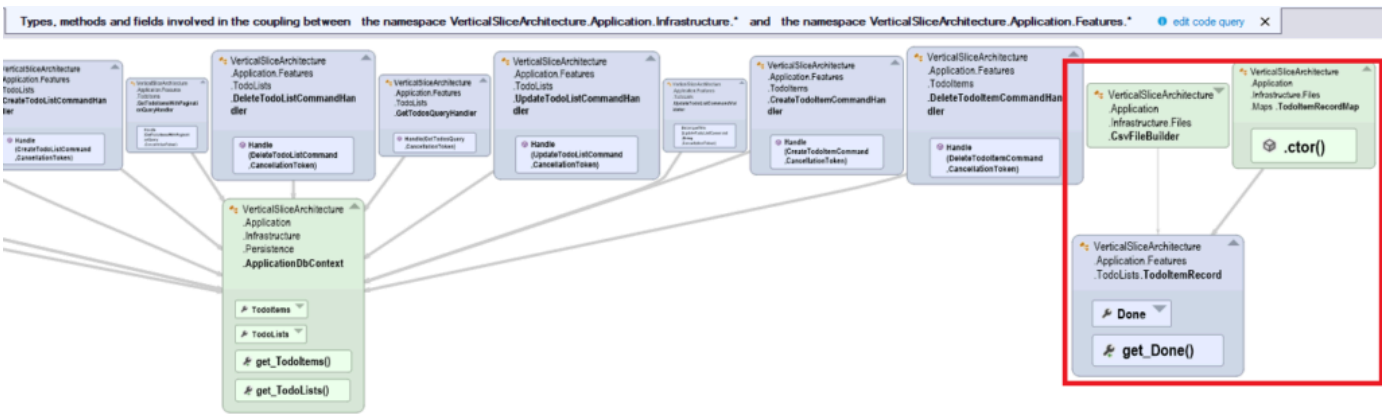
Note on this implementation

The dependency diagram above contains two red arrows that reveal that `Application.Common` and `Application.Infrastructure` both rely on `Feature` which I find somewhat frustrating. By double-clicking both arrows we can obtain two coupling graphs. Each of these graphs sheds light on the fact that `CsvFileBuilder` consuming `TodoItemRecord` is the culprit in both cases. IMHO `CsvFileBuilder` should be abstracted from the features implementations.

Why `Application.Common` is relying on `Feature` :



Why `Application.Infrastructure` is relying `Feature` :



Download the NDepend Trial and gain valuable insights into your .NET code within a few minutes

for improving code maintainability, security, and overall quality. Whether you're modernizing a legacy .NET application or starting fresh in C#, get started with your free full-featured trial today!

Make your .NET code beautiful with NDepend

Download the NDepend Trial and gain valuable insights into your .NET code within a few minutes

DOWNLOAD THE **14 DAY TRIAL** NOW

Download Free
NDepend Trial



Get informed about our latest blog post.

Type your email... SUBSCRIBE

Download the NDepend Trial and gain valuable insights into your .NET code within a few minutes

Architecture	42
C#	67
Case Study	7
NDepend	22
Performance	8
Visual Studio	60

Recent Posts

- Pointers in C# and Memory Safety: Span vs. C# 16 unsafe June 3, 2026
- Maintainability Index: A Practical Guide for C# Developers June 1, 2026
- Improve C# code performance with Span<T> June 1, 2026
- C# Nullable Types Explained (2026) June 1, 2026
- Anatomy of an Open-Source AI Coding Agent Built in .NET: CodeAlta May 31, 2026

Download the NDepend Trial and gain valuable insights into your .NET code within a few minutes

Trending

Dependency Graph

Dependency Matrix

Code Diff

Default Code Rules

NDepend API

Copyright © 2004-2023 ZEN PROGRAM

Made with by Growth Labs